LỘ TRÌNH ĐÀO TẠO FRESHER JAVA

Ngày	Nội dung đào tạo	Link tài liệu	Kết quả
Tuần 1 – Tuần 2: Tìm hiểu về quy trình của công ty và ngôn ngữ Java			
Ngày 1	1. Nhận máy và Setup account 2. Giới thiệu và làm quen với mọi người và văn hoá công ty 3. Training Security		1. Install xong máy và các ứng dụng cơ bản, setup xong account 2. Pass security exam
Ngày 2	1. Tìm hiểu về GIT 2. Đọc về Java coding convention	1. Git 2. Java_Coding_Convention.xls	3. Nắm được các lệnh về GIT, Xây dựng được gitflow 4. Nắm được các yêu cầu trong java coding convention và áp dụng cho các bài tập hoặc demo.
Ngày 3 (0.5 ngày cho setup)	1. Setup jdk 17 2. Setup Spring tools 3. MySQL server	1. https://www.oracle.com/java/technologies/javase/jdk17-archive-downloads.html 2. https://spring.io/tools 3. https://dev.mysql.com/downloads/windows/installer/5.7.html	
Ngày 3 – Ngày 15	Tìm hiểu Java core 1. Tìm hiểu và sử dụng được những thư viện trong gói java.util ► List và các thư viện đi kèm theo List	1. https://www.javatpoint.com/java-tutorial 2. https://www.tutorialspoint.com/java8/index.htm	Làm dạng slide những cái đã tìm hiểu + demo

	► Map và các thư viện đi kèm theo Map, HashMap, HashTable ► Set và các thư viện đi kèm theo Set ► Date ► Collections ► Vectors ► Properties ► Resource Bundle ► Calendar ► Enum ► Iterator Tìm hiểu sự khác nhau của từng loại cấu trúc dữ liệu, Ưu và nhược điểm của từng loại Tìm hiểu cách xử lý cho từng collection trong môi trường multithread 2. Tìm hiểu và sử dụng được những thư viện trong gói java.io ►FileInputStream ►FileOutputStream ► BufferedInputStream ► BufferedOutputStream + Read and write for files: CSV, TxT, Excel. (Sử dụng thư viện) + Export PDF. (Tạo template thống kê động (HTML.vv..) và tự động điền data vào template sau đó xuất ra PDF) 3. Khi nào sử dụng String/ String Builder/ String Buffer		

	4. Java 8 ► Lambda Expression ► Functional Interfaces ► Method references ► Optional (Tìm hiểu toàn bộ method của optional) ► Default Methods ► Streams, ParallelStreams API (sử dụng: duyệt list data, convert List to Map, tìm min và max của list trong object, ….) ► Date and Time APIs		
Ngày 16 – Ngày 20	1. Tìm hiểu về OOP Tại sao cần dùng OOP → Lợi ích (điểm mạnh) khi dùng OOP → Các đặc tính của OOP → Nắm vững được các đối tượng liệt kê dưới đây : ► Object ► Class ► Inheritance ► Interface ► Package ► Modifiers ► Overloading vs overiding → Khi nào sử dụng Interface, khi nào sử dụng abstract class. Nêu điểm mạnh của từng loại 2. Tìm hiểu về Design Pattern	1. https://www.javatpoint.com/java-oops-concepts 2. https://www.digitalocean.com/community/tutorials/java-design-patterns-example-tutorial	Làm dạng slide những cái đã tìm hiểu + demo

	Tại sao cần áp dụng Design Pattern trong việc thiết kế, sản xuất phần mềm? → Đặc tính của Design Pattern → Có bao nhiêu loại (category) Design Pattern, kể tên từng loại. → Liệt kê những Design Pattern mà bạn biết trong từng loại (category) - sơ đồ hoạt động		
Tuần 5 – 6 – 7 – 8 – 9 – 10- 11 -12: Tìm hiểu về Spring Framework			
Ngày21 - ngày 23	**1.** Tìm hiểu về Spring Framework → Tìm hiểu về kiến trúc (architecture) của Spring FW. → Điểm mạnh của Spring FW. 2. Tìm hiểu về các core component của Spring Framework → Inversion Of Control (IoC) and Dependency Injection (DI). → Aspect Oriented Programming (AOP). 3. Tìm hiểu về các thành phần khác của Spring Framework → Spring - Bean Life Cycle. → Spring - Injecting. → Spring - Beans Auto-Wiring	1. https://www.javatpoint.com/spring-tutorial	Tìm hiểu kỹ và nắm được kiến thức nền. (Không cần làm demo, nhưng phải nắm được kiến thức khi mentor hỏi)
Ngày 24-40	1. Tìm hiểu basic annotation → @SpringBootApplication → @Component → @Controller, @RestController,		1. Nắm kiến thức 2. Làm bài tập số 1

	@ResponseBody → @Service → @Repository- @Configuration → @Bean @Lazy, @Primary, @Qualifier → @Autowired → @Value → @Transactional, @Query, @Modifying → @RequestParam, @RequestBody, @PathVariable, @RequestHeader → @RequestMapping, @PostMapping, @GetMapping, @PutMapping, @DeleteMapping → Annotations use validate input data @Valid, @NotNull, @Nullable, @Length,... → Annotations of lombok 2. Communicating with databases → Hibernate → JPA → Understanding mapping entities with tables in the database 3. Spring Security + JWT(login, logout ,call API after loged in, phần quyền cho API theo role) 4. Filter, Filterchain, CORS, Request, Response, Session, CSRF		

	5. Search → Criteria and query Builder → paging → Sort multi conditions → Specification → Native query 6. Configure profiles for Spring Boot application 7. Log configuration 8. Junit test 9. Docs API(Spring OpenAPI)		
Ngày 41-50	11. Microservices(Eureka, FeignClient) 12. Load balancer 13. API Gateway (zuul) 14. WebSocket 15. Kafka 16. Upload/download File, Multipart 17. Scheduler 18. Rest template 19. Build Dockerfile, docker-compose for Spring Boot Application		1. Nắm kiến thức Làm bài tập số 2 **(**
Tuần 11: Các vấn đề thường gặp trong dự án			
Ngày 51-55	1. Exception và cách sử dụng try/catch/finally → Exception là gì ? → Catch và handle exception		Tái hiện và handle được, demo

	→ Khi nào nên sử dụng try-catch-finally → Throw exception → Tạo lớp exception riêng → Ưu điểm của việc sử dụng exception 2. Transaction khi xử lý Insert/Update/Delete database → Viết demo sử dụng CRUD để thao tác dữ liệu, trường hợp có lỗi thì rollback transaction (kiểm tra để chắc chắn rằng dữ liệu không được update vào database), trường hợp không có lỗi thì thực hiện commit. → Làm demo trong trường hợp có nhiều xử lý INSERT/ UPDATE/ DELETE, chỉ cần 1 thao tác thất bại, tất cả phải được rollback. Trường hợp thành công hết cả 3 mới được update dữ liệu vào database. + Sử dụng @Transaction để quản lý transaction → Viết demo sử dụng CRUD để thao tác dữ liệu, trường hợp có lỗi thì rollback transaction (kiểm tra để chắc chắn rằng dữ liệu không được update vào database), trường hợp không có lỗi thì thực hiện commit. → Làm demo trong trường hợp có nhiều xử lý INSERT/ UPDATE/		

	DELETE, chỉ cần 1 thao tác thất bại, tất cả phải được rollback. Trường hợp thành công hết cả 3 mới được update dữ liệu vào database. 3. Security (cách phòng chống SQL Injection và XSS) + SQL Injection → SQL Injection là gì ? Tác hại ra sao ? → Viết demo 1 chương trình đơn giản gồm chức năng login và 1 form search dữ liệu. Sử dụng kỹ thuật SQL Injection để pass qua bước đăng nhập, Sau đó tiếp tục dùng SQL Injection để khai thác dữ liệu trong database (nhập data vào form search và dùng SQL Injection để query dữ liệu trong database). → Cách khắc phục lỗi SQL Injection ? Tiến hành phòng chống lỗi SQL Injection ở demo trên. + XSS → XSS là gì ? Tác hại ra sao ? → Viết demo chương trình đơn giản gồm form nhập dữ liệu và màn hình hiển thị dữ liệu. Sử dụng kỹ thuật XSS để tấn công và insert dữ liệu xuống database. Khi đi vào màn hình hiển thị dữ liệu, lúc load dữ liệu từ database lên sẽ bị dính lỗi XSS (có thể chọn cách đơn		

	giản như alert trên javascript, một button hoặc link dẫn người dùng tới một trang giả mạo khác, .v.v). → Cách khắc phụ lỗi XSS ? Tiến hành phòng chống lỗi XSS ở demo trên		

Bài Tập 1: (Tham khảo, cần đảm bảo các yêu cầu trong file Chấm Điểm Training)

Sử dụng Spring boot + JPA để xây dựng một ứng dụng quản lý thư viện(chỉ cần viết API, nếu viết được web càng tốt sẽ đánh giá cao).

Ứng dụng gồm có các chức năng như sau :

→ Đăng nhập vào hệ thống bằng tài khoản admin (giả định tài khoản admin là admin/admin).

→ Logout

→ Đăng kí member mới (email, pass)

→ Hệ thống có 2 role: Admin(Quản lý sách, Quản lý member, Tạo member, ...), User(Mượn trả sách, change password, reset pass, change mail đăng kí). Mỗi role chỉ được phép gọi vào những API mình có quyền

→ Màn hình quản lý Sách

- API search
- API thêm/xóa/sửa
- API import sách mới bằng file csv

→ Màn hình quản lý Member

- Tìm kiếm thành viên của thư viện theo các tiêu chí, học viên tự nghĩ thêm**(càng nhiều càng tốt).** Sau đó hiển thị lên table.

+ Theo tên (search like), theo tên sách, ...

+ Theo ngày tháng năm sinh(hiển thị dialog để chọn ngày format yyyy/MM/d điểm). Chú ý là tìm trong khoảng Date from đến date to. Check validation khi nhập ngày không đúng format.

+ Theo sách, thực hiện load dữ liệu lên combobox để người dùng có thể chọn.

=> Lưu ý :Phải search trên 4, 5 điều kiện

- Tạo mới member, update thông tin member

→ Màn hình mượn sách(API Mượn sách validate số lượng sách trong kho đủ cho mượn, mỗi người chỉ được phép mượn 1 loại sách)

→ Màn hình trả sách(API trả sách)

→ Màn hình config hệ thống(API tắt bật maintenance mode)

** Lưu Ý: Tất cả các API tạo, update đều phải validate.

Yêu cầu và điểm cho từng phần:

→ Hoàn thành các chức năng nêu trên.

- ➔ Login: Yêu cầu: sử dụng Spring Security + JWT
- ➔ Logout: yêu cầu khi logout phải chặn không cho sử dụng token đã logout
- ➔ Đăng kí: yêu cầu phải validate đúng format email và pass. Khi người dùng đăng kí thành công sẽ send verify mail tới email đã đăng kí. Khi user click vào link verify trong mail sẽ trỏ tới hệ thống và ghi nhận user đã verify mail thành công. Account này mới có thể login vào hệ thống, nếu trường hợp chưa verify mail thì cần phải thông báo cho user biết khi login.
- ➔ Màn hình reset pass(API reset pass)
- ➔ Màn hình change pass(API change pass)
- ➔ Màn hình change mail(API change mail: khi thay đổi mail mới sẽ code verify cho mail mới, nhập đúng code verify sẽ cho phép user change mail)
- ➔ Màn hình quản lý Sách

  + API search: lấy đúng thông tin và có phân trang, mỗi trang tối đa 10 records , Nâng cao: Nếu sử dụng Spring Data Specifications

  + API thêm/xóa/sửa: Viết theo dạng chuẩn RestFul API và validate dữ liệu đầu vào

  + API import sách mới: Yêu cầu phải có validate file size nhỏ hơn 5MB, chỉ hỗ trợ file csv.
- ➔ Màn hình quản lý Member: yêu cầu và điểm như Màn hình quản lý Sách
- ➔ Màn hình mượn sách:
- ➔ Màn hình trả sách:
- ➔ Màn hình config hệ thống: (Viết API tắt bật maintenance mode và API hiển thị config hiện tại, Lưu ý: khi bật maintenance mode thì cần chặn tất cả API gọi lên hệ thống)

→ Coding sạch, đẹp, comment rõ ràng, tuân thủ Java Coding Convention**.**

→ Phải sử dụng các maping relationship table của hibernate như One-to-One, One-to-Many, Many-to-Many

→ **Bắt buộc sử dụng lombok**
→ Xử lý CRUD phải sử dụng transaction (để rollback trong trường hợp có lỗi)
→ Sử dụng log4j để đặt log cho tất cả các chức năng(Request và Response), log console và log file.
→ Handle exception trước khi trả về cho client
→ Sử dụng Aspect Oriented Programming (AOP) : ví dụ xử lý cho chức năng chứng thực, hoặc ghi log, v.v.
→ Xử lý trường hợp refesh token **-> yêu cầu nếu trường hợp refresh token còn hạn thì sẽ tự động tạo token mới cho user thay vì báo 401 và bắt user login lại**
→ Viết Junit test cho các API đã làm
→ Viết migration database sử dụng Liquibase
→ Viết Doc API

Bài tập 2: (Tham khảo, cần đảm bảo các yêu cầu trong file Chấm Điểm Training)
Cùng yêu cầu bài tập 1 nhưng sẽ làm theo dạng microservices
→ Tạo discovery-service
→ Tạo gateway-service
→ Tạo auth-service (Thực hiện login và logout)
→ Tạo user-service (Quản lý thông tin của account, thông tin của member)
→ Tạo book-service (Quản lý thông tin sách và thông tin member mượn sách)

Yêu cầu:
→ Đảm Bảo các tính năng như trên bài tập 1.
→ Các service giao tiếp với nhau sử dụng FeignClient, RestTemplate.
→ 2 service: user và book kết nối với 2 schema khác nhau
→ Load Balancer.
→ Sử dụng Zuul gateway
→ Tạo thêm socket-service(Xử lý các việc realtime của ứng dụng). Tạo tính năng khi member mượn/trả sách gửi notification cho admin.
Yêu cầu nâng cao
→ Sử dụng Vault by HashiCorp cho các config quan trọng như user/pass của DB.
→ Sử dụng zipkin để track request đã đi qua những service nào.
→ Chức năng login bằng facebook hoặc google

→ Chức năng login with 2Factor Authentication